

01_variables-datatypes

August 19, 2026

1 01 - PYTHON VARIABLES & DATATYPES

Name: Vamshi / Date: 05/08/2026

- Variable is a name assigned to a value. This name refers to a memory location.
- Python variables are also called as objects.
- Syntax: **variable = value**

variable —> name assigned for memory location & value —> datatype

Rules to define a python variable name:

1. Variable names must ****start with a letter or an underscore (_)****.
2. They **cannot start with a digit/number** but can end with digit/number.
3. It can contain ****letters, digits, and underscores (_)**** but, only if above 2 rules are met.
4. **Cannot contain spaces or special characters.**
5. They are **case-sensitive** (python, Python, and PYTHON are different variables).
6. It **cannot be Python keywords.** =====> Keywords are also known as reserved words.

Comments in Python: - Single line comments- Use **# comment** - Multi line comments- Use “ “ ” **comment “ “ ” or ’ ’ ’ comment ’ ’ ’** or above method can be used for each line

```
[5]: n=10 # Integer value
      n
```

```
[5]: 10
```

```
[6]: 10=n # Not Valid - syntax error - Rule 2
```

```
Cell In[6], line 1
    10=n # Not Valid - syntax error
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='
```

```
[8]: name = 'stark' # String value
      name
```

```
[8]: 'stark'
```

```
[9]: NAME # Name Error - Rule 5, becoz variable NAME is not present (its case_
      ↪sensitive - name and NAME are not the same)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 NAME #

NameError: name 'NAME' is not defined
```

```
[10]: 1a = 25 # Syntax Error - Rule 2
```

```
Cell In[10], line 1
    1a = 25
    ^
SyntaxError: invalid decimal literal
```

```
[11]: name@ = 'tony' # Syntax Error - Rule 4
      name@
```

```
Cell In[11], line 1
    name@ = 'tony' # Name Error - Rule 4
    ^
SyntaxError: invalid syntax
```

```
[12]: name_1 = 'tony' # Rule 3
      name_1
```

```
[12]: 'tony'
```

```
[13]: for = 45 # Syntax Error - Rule 6 (kw cannot be used as variable name)
```

```
Cell In[13], line 1
    for = 45 # Rule 6
    ^
SyntaxError: invalid syntax
```

```
[15]: FOR = 45
      FOR
```

[15]: 45

```
[23]: val = 53.45 # Float value
      val
```

[23]: 53.45

```
[33]: val_1 = True # Boolean value
      val_1
```

[33]: True

To display the output on the screen, print() function is used.

```
[19]: print(FOR)
      print(name_1)
      print(name)
      print(n)
```

45
tony
stark
10

****The type() function is used to identify the datatype of the variable****

```
[35]: print(type(n))
      print(type(name))
      print(type(val))
      print(type(val_1))
```

<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>

To check the list of all keywords and print total number of keywords:

```
[4]: import keyword
      keyword.kwlist # Lists all the key words
```

[4]: ['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',

```

'continue',
'def',
'del',
'elif',
'else',
'except',
'finally',
'for',
'from',
'global',
'if',
'import',
'in',
'is',
'lambda',
'nonlocal',
'not',
'or',
'pass',
'raise',
'return',
'try',
'while',
'with',
'yield']

```

```

[6]: kw = len(keyword.kwlist) # len() function is used get total num of items/length
    ↪ of an object
print("The total number of keywords are", kw)

```

The total number of keywords are 35

```

[17]: first_name = 'VAMSHI'
last_name = ' '
country = 'INDIA'
city = 'Bangalore'
age = 22
is_married = False
skills = ['Python', 'AI', 'Robotics', 'C++', 'Embedded Sys']
person_info = {
    'firstname': 'Mowgli',
    'lastname': ' ',
    'country': 'Jungle',
    'city': 'Near the riverside'
}

# Printing all the Values

```

```

print('First name:', first_name)
print('First name length:', len(first_name))
print('Last name: ', last_name)
print('Last name length: ', len(last_name))
print('Country: ', country)
print('City: ', city)
print('Age: ', age)
print('Married: ', is_married)
print('Skills: ', skills)
print('Person information: ', person_info)

```

```

First name: VAMSHI
First name length: 6
Last name:
Last name length: 1
Country: INDIA
City: Bangalore
Age: 22
Married: False
Skills: ['Python', 'AI', 'Robotics', 'C++', 'Embedded Sys']
Person information: {'firstname': 'Mowgli', 'lastname': ' ', 'country':
'Jungle', 'city': 'Near the riverside'}

```

[18]: *# Declaring multiple variables in one line*

```

first_name, last_name, country, age, is_married = 'vam', 'abc', 'ind', 22, False

print(first_name, last_name, country, age, is_married)
print('First name:', first_name)
print('Last name: ', last_name)
print('Country: ', country)
print('Age: ', age)
print('Married: ', is_married)

```

```

vam abc ind 22 False
First name: vam
Last name: abc
Country: ind
Age: 22
Married: False

```

Name: Vamshi / Date: 06/08/2026 `id()` function returns the address of the memory location

[7]: *# Example 1*

```

a = 101
id(a)

```

[7]: 140721251964472

```
[ ]: # Example 2 --> a1 and a2 have diff values therefore, the address of both the
      ↪ variables is diff
a1, a2 = 102, 201
print(id(a1))
print(id(a2))
```

140721251964472

140721251967672

```
[10]: # Example 3 --> x1 and x2 have same values therefore, both the variables are
      ↪ pointing to same address
x1, x2 = 2, 80
print(id(x1))
print(id(x2))
```

140721251961304

140721251963800

- Functions without arg: fruit()
- Functions with arg: fruits(with seed, without seed) —> it can have multiple arguments

True —> 1 & False —> 0 in boolean operation

```
[12]: b = True      # represents 1
      b1 = False    # represents 0

print(b+b1) # gives 1 as O/P
print(b1/b) # gives 0.0 as O/P
print(b1//b) # gives 0 as O/P
print(b*b1) # gives 0 as O/P
```

1

0.0

0

0

```
[14]: print(b/b1) # Invalid ---> Zero Division Error
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 print(b/b1) # Invalid ---> Zero Division Error

ZeroDivisionError: division by zero
```

Complex Datatype: $a + bj$ form - 'a' is real part - 'b' is imaginary part

The complex data type is used in mathematical, scientific computing, engg. simulations etc....

```
[15]: c = 50 + 30j
      c
```

```
[15]: (50+30j)
```

```
[16]: c1 = 50 + 70j
      c1
```

```
[16]: (50+70j)
```

```
[18]: c+c1 # Adds real part of c with real part of c1 and imaginary part of c with
      ↪imaginary part of c1
```

```
[18]: (100+100j)
```

To get the real and imaginary part:

```
[ ]: print(c)      # complex data
      print(c.real) # real part
      print(c.imag) # imaginary part
```

Name: Vamshi / Date: 07/08/2026 (Strings - Indexing & Slicing)

```
[22]: s = welcome # this is not a string value becoz it must be specified within ' '
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[22], line 1
----> 1 s = welcome # this is not a string value becoz it must be specified
      ↪within ' '

NameError: name 'welcome' is not defined
```

```
[2]: s = welcome here
```

```
Cell In[2], line 1
      s = welcome here
          ^
SyntaxError: invalid syntax
```

```
[3]: s = 'welcome'
      s
```

```
[3]: 'welcome'
```

```
[4]: s1 = "welcome"
s1
```

```
[4]: 'welcome'
```

```
[5]: s2 = '''welcome'''
s2
```

```
[5]: 'welcome'
```

```
[7]: s3 = 'welcome to datatypes of python
      I am learning strings now'
s3
```

```
Cell In[7], line 1
      s3 = 'welcome to datatypes of python
      ~
SyntaxError: unterminated string literal (detected at line 1)
```

- We make use of ''' ''' for a multi line string data

```
[11]: s4 = '''welcome to datatypes of python
      I am learning
      strings now'''
s4
```

```
[11]: 'welcome to datatypes of python \n I am learning \n strings now'
```

String Indexing

- Types:
 - Forward Indexing
 - Backward Indexing
 - Step Indexing

```
[3]: st = 'robots'
st
```

```
[3]: 'robots'
```


2 [0 1 2 3 4 5] —> This is Forward Index (left to right)

3 ['r' 'o' 'b' 'o' 't' 's']

4 [-6 -5 -4 -3 -2 -1] —> This is Backward Index (right to left)

Note: Python Index always starts with 0

```
[13]: st[0]
```

```
[13]: 'r'
```

```
[25]: st[1]
```

```
[25]: 'o'
```

```
[26]: st[2]
```

```
[26]: 'b'
```

```
[24]: st[3]
```

```
[24]: 'o'
```

```
[27]: st[4]
```

```
[27]: 't'
```

```
[14]: st[5]
```

```
[14]: 's'
```

```
[23]: st[6] # Not valid becoz the index is not present
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[23], line 1  
----> 1 st[6] # Not valid becoz the index is not present  
  
IndexError: string index out of range
```

The following is reverse indexing:

```
[4]: st[-1]
```

```
[4]: 's'
```

```
[5]: st[-2]
```

```
[5]: 't'
```

```
[6]: st[-3]
```

```
[6]: 'o'
```

```
[7]: st[-4]
```

```
[7]: 'b'
```

```
[8]: st[-5]
```

```
[8]: 'o'
```

```
[9]: st[-6]
```

```
[9]: 'r'
```

String Slicing is used to slice or get a specific part of a string.

```
[2]: string = 'This is a test code'
     string[:] # it returns the entire string
```

```
[2]: 'This is a test code'
```

```
[29]: string[0:4] # it returns from index 0 to index 3 ---> (n-1) where 'n' is the
      ↪ stop index which is excluded
```

```
[29]: 'This'
```

```
[30]: string[4:] # it starts from index 4 and gives the entire string till the end
      ↪ becoz stop index is not specified
```

```
[30]: ' is a test code'
```

```
[31]: string[:10] # start index not specified so it starts from zero and gives the 0/
      ↪ P till (n-1) i.e 9
```

```
[31]: 'This is a '
```

```
[11]: string[-4:] # retrieves the last 4 characters of the string
```

```
[11]: 'code'
```

```
[ ]: string[:-1] # excludes the value at index -1
```

```
[ ]: 'This is a test cod'
```

```
[18]: string[::1]
```

```
[18]: 'This is a test code'
```

```
[ ]: string[::-2]
```

```
[ ]: 'Ti sats oe'
```

```
[4]: string[::-1] # reverse
```

```
[4]: 'edoc tset a si sihT'
```

```
[5]: string[::-2]
```

```
[5]: 'eo stas iT'
```

These are some Case Operations

```
[ ]: print(string.capitalize()) # First letter captialized
     print(string.lower())      # all lower case
     print(string.swapcase())   # Upper case becomes lower case and lower case
     ↪ becomes upper case
     print(string.title())      # Every first letter of a word ---> Capital/Upper
     ↪ case
     print(string.upper())      # Entire string gets captalized
```

```
This is a test code
this is a test code
tHIS IS A TEST CODE
This Is A Test Code
THIS IS A TEST CODE
```

```
[3]: len(string) # length of the string
```

```
[3]: 19
```

```
[38]: stg1 = 'Hooray!!!! '
     print(stg1)

     stg1 = stg1 * 10 # multiplies stg1 10 times
     print(stg1)
```

```
Hooray!!!!
Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!!
Hooray!!!! Hooray!!!! Hooray!!!!
```

```
[39]: # String Concatenation

     n1 = 'Hi, '
     n2 = 'This is Bob.'
     n1+n2 # It combines both the strings together
```

```
[39]: 'Hi, This is Bob.'
```

```
[ ]: str1 = "Hi, Charlie"  
print(str1)  
str1[0:2] = "Oh" # Strings are Immutable ----> value cannot be changed
```

Hi, Charlie

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[5], line 3  
      1 str1 = "Hi, Charlie"  
      2 print(str1)  
----> 3 str1[0:2] = "Oh"  
  
TypeError: 'str' object does not support item assignment
```

```
[ ]: del n1 # it deletes the string --> so the variable doesnt exist in memory  
      ↳ therefore, name error occurs  
n1
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[22], line 1  
----> 1 del n1 # it deletes the string --> so the variable doesnt exist in  
      ↳ memory so, name error occurs  
      2 n1  
  
NameError: name 'n1' is not defined
```

Name: Vamshi / Date: 10/08/2026 (Type casting, print, end statement and separator)

```
[6]: int(120.34) # float to int
```

[6]: 120

```
[7]: int(True) # bool to int
```

[7]: 1

```
[9]: int('25') # number-string to int
```

[9]: 25

```
[10]: int('I am Groot') # text-string to int ----> not possible
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 1
----> 1 int('I am Groot') # text-string to int ----> not possible

ValueError: invalid literal for int() with base 10: 'I am Groot'
```

```
[ ]: int(4+10j) # complex to int ----> not possible
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 int(4+10j)

TypeError: int() argument must be a string, a bytes-like object or a real_
↳number, not 'complex'
```

Therefore, All datatypes (except Text-String & Complex) can be converted to int

```
[11]: float(3402) # int to float
```

```
[11]: 3402.0
```

```
[12]: float(False) # bool to float
```

```
[12]: 0.0
```

```
[13]: float('99') # number-string to float
```

```
[13]: 99.0
```

```
[14]: float('xyz') # text-string to float ----> not possible
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 float('xyz') # text-string to float ----> not possible

ValueError: could not convert string to float: 'xyz'
```

```
[15]: float(2+9j) # complex to float ----> not possible
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[15], line 1
```

```
----> 1 float(2+9j) # complex to float ----> not possible
```

```
TypeError: float() argument must be a string or a real number, not 'complex'
```

Therefore, All datatypes (except Text-String & Complex) can be converted to float

```
[16]: bool(10) # int to bool
```

```
[16]: True
```

```
[17]: bool(0) # 0 value always returns false
```

```
[17]: False
```

```
[18]: bool(5.8) # float to bool
```

```
[18]: True
```

```
[19]: bool(9+12j) # complex to bool
```

```
[19]: True
```

```
[20]: bool('101') # number-string to bool
```

```
[20]: True
```

```
[21]: bool('abc') # text-string to bool
```

```
[21]: True
```

```
[23]: bool("") # empty
```

```
[23]: False
```

Therefore, All datatypes can be converted to bool (Every non-zero values returns True)

```
[24]: complex(30) # int to complex
```

```
[24]: (30+0j)
```

```
[25]: complex(99.99) # float to complex
```

```
[25]: (99.99+0j)
```

```
[26]: complex(30,33) # real & imaginary part - int
```

```
[26]: (30+33j)
```

```
[27]: complex(99.99,87.4) # real & imaginary part - float
```

```
[27]: (99.99+87.4j)
```

```
[28]: complex(True) # bool to complex
```

```
[28]: (1+0j)
```

```
[29]: complex(False) # bool to complex
```

```
[29]: 0j
```

```
[31]: complex(True, True)
```

```
[31]: (1+1j)
```

```
[30]: complex('50') # number-string to complex
```

```
[30]: (50+0j)
```

```
[32]: complex('hundred') # text-string to complex ----> not possible
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[32], line 1  
----> 1 complex('hundred') # text-string to complex ----> not possible  
  
ValueError: complex() arg is a malformed string
```

```
[34]: complex('10', '20') # only one string parameter valid
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[34], line 1  
----> 1 complex('10', '20') # only one string parameter valid  
  
TypeError: complex() can't take second arg if first is a string
```

Therefore, All datatypes (except Text-String) can be converted to complex

```
[35]: str(100) # int to string
```

```
[35]: '100'
```

```
[36]: str(100.0) # float to string
```

```
[36]: '100.0'
```

```
[37]: str(True) # bool to string
```

```
[37]: 'True'
```

```
[38]: str(8+8j) # complex to string
```

```
[38]: '(8+8j)'
```

Therefore, All datatypes can be converted to string

`print()` is used to display the output or a line of code on the screen.

```
[1]: n1 = 10
      n2 = 30
      print() # without argument ---> No Output
      print(n1) # with one or n-arguments
      print("The addition of",n1,"&",n2,"is",n1+n2)
```

```
10
```

```
The addition of 10 & 30 is 40
```

```
[2]: print("The addition of {} and {} is {}".format(n1,n2,n1+n2)) # with format_
      ↪method
```

```
The addition of 10 and 30 is 40
```

```
[4]: print(f"The addition of {n1} and {n2} is {n1+n2}") # f-string method
```

```
The addition of 10 and 30 is 40
```

The `end` statement is used to join end to statement-1 with the start of statement-2

```
[10]: print("Hi")
      print("Everyone")

      print("Hi",end=" ") # Statement-1
      print("Everyone") # Statement-2
```

```
Hi
```

```
Everyone
```

```
Hi Everyone
```

If any value is specified with `end` then, it will be used to join the statements

```
[11]: print("Hi",end="2") # Statement-1
      print("Everyone") # Statement-2
```

```
Hi2Everyone
```

The default separator used is space. We can use any character as a separator

```
[14]: print("Hi", "Everyone", "This", "Way", " ", sep="----->")
```

Hi----->Everyone----->This----->Way----->

```
[15]: name = "naresh"  
print(name, "gmail.com", sep="@")
```

naresh@gmail.com

```
[16]: print("Python", "Java", "C++", sep=" | ")
```

Python | Java | C++

```
[18]: print("Me", "Good", sep="", end="")  
print("Dream")
```

Here, sep="" removes spaces and end="" prevents a newline

MeGoodDream

```
[19]: print("31", "10", "1991", sep="-")
```

31-10-1991